

Frameworks and Architectures for the Web: Building a Software System for Web Shopping

2026-05-29

Group 9

Sine Ergin	Cecilie Malmberg	Melanie Puck	Miguel A. Castro G
sier@itu.dk	cecm@itu.dk	mepu@itu.dk	migca@itu.dk

Characters including spaces: 14,683 (6 pages)

Contents

1	Web design of the client-side application	3
1.1	Information architecture	3
1.2	Taxonomies and product information	3
1.3	Site navigation and page hierarchy	3
1.4	Interface design, responsive layout and registration validation .	6
2	Design of the RESTful API	6
2.1	Login and Registration endpoints	6
2.2	Email and Username for login and registration	7
2.3	Backend services	7
3	Software architecture of the developed system	8
3.1	Logical and physical architecture	8
3.1.1	Logical architecture	8
3.1.2	Physical architecture	9
3.1.3	Third-party components and frameworks	10
3.2	Client-side React application architecture and patterns	10
3.2.1	Single-page application (SPA) architecture	11
3.2.2	State management patterns	11
3.2.3	Custom hooks and separation of logic	12
3.2.4	Form handling and validation	12
3.2.5	Responsive design principles	12
3.3	Server-side RESTful API architecture and patterns	12
3.3.1	Layered architecture	12
3.3.2	Data storage solution	13
3.3.3	Modular backend design	13
4	Use of generative AI in the project	13
A	API Documentation	14

1 Web design of the client-side application

The design of the webshop continues the visual concept introduced in Mini-project 1: a simple clothing shop focused on t-shirts. The change lies in the interface now having been implemented as a React application in TypeScript, with product and basket information from the server-side RESTful API.

1.1 Information architecture

The webshop offers a focused catalogue of 15 t-shirts and the site hierarchy follows a simple shopping journey. The home page introduces current offers and new arrivals, the product catalogue supports browsing and filtering, the product detail view presents information for one selected product, and the basket view supports the purchasing decision. A customer can shop as a guest, but can also register as a user and log in.

1.2 Taxonomies and product information

Two product taxonomies are used to browse the products: color and gender. The available color options consist of: red, orange, yellow, green, blue, purple, black and white. Furthermore, it's possible to pick between Men and Women for gender options.

The filter allows combinations across taxonomies. For example, selecting Blue and Men returns only products that satisfy both choices. Meanwhile, it's possible to select multiple different colors to show at the same time.

Each t-shirt product has the same information properties. These include a name, color, gender, price, discount, image, description, and whether it's a new arrival.

1.3 Site navigation and page hierarchy

The website has 6 pages:

- The Home Page
- The All Products Page
- The Product Detail Page
- The Basket Page
- The Login Page

- The Registration Page.

The **Home page** displays products that are currently on sale and products that are new arrivals with the option to add each product to the basket. The **All Products page** displays the 15 available products in a card format also with the option to add a product to the basket. The **Product Detail page** shows detailed information about a specific product and also allows the user to add the product to the basket. The **Basket page** shows the user's basket with the option to increase or decrease the quantity of each product and place an order. The **Login page** allows the user to log in or register as a user. If the user wants to register, they can navigate to the **Registration page**. It is not required to be a registered and logged-in user to browse the website or to add products to the basket and make an order. If the user is logged in, the user's name will be visible on all pages, written on the navbar.

Each page has a navigation bar at the top that allows the user to navigate to the key pages. These are the **Home page**, the **All Products page**, the **Basket page**, and the **Login page**. Figure 1 below shows a wireframe of the site structure.

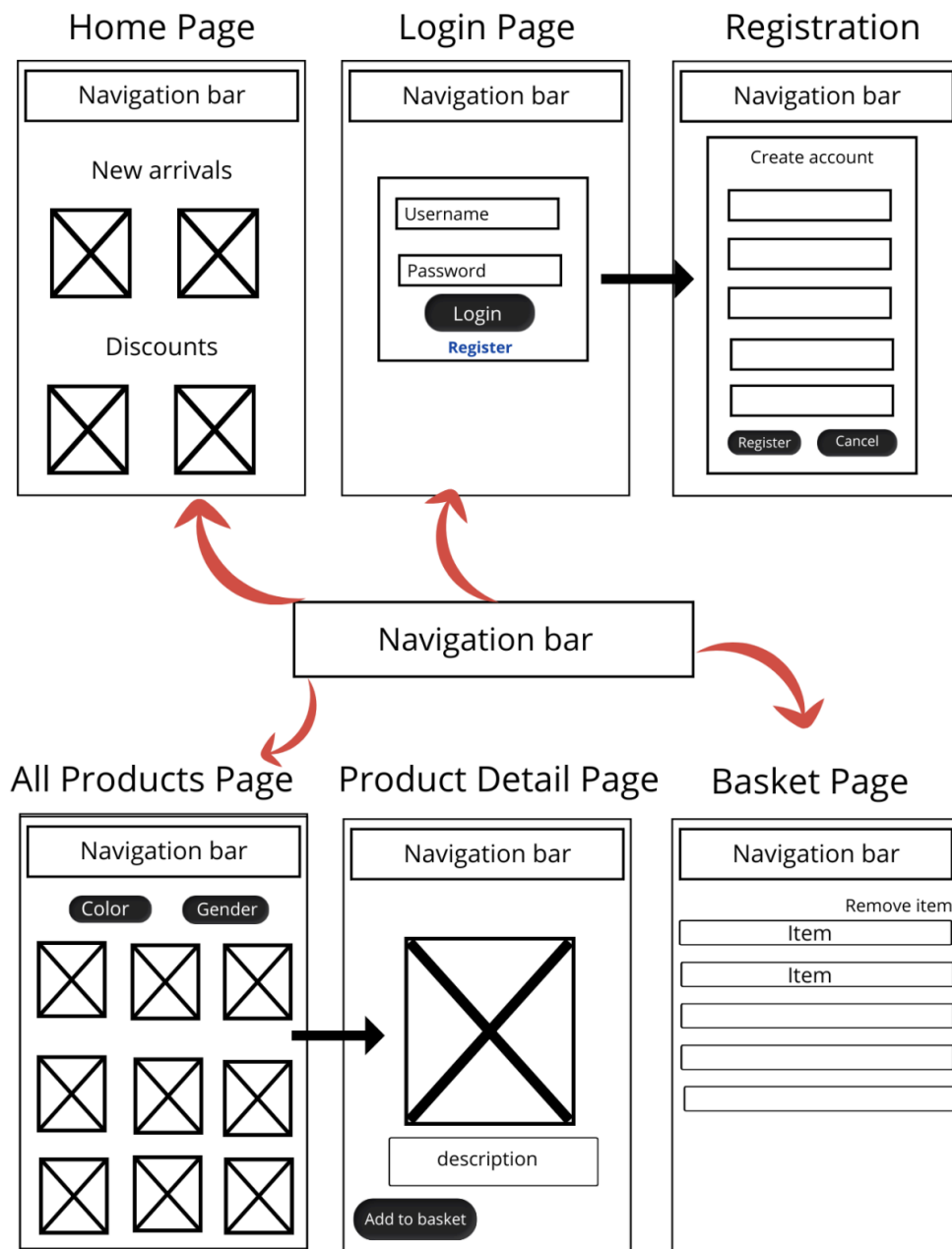


Figure 1: Wireframe of the website

1.4 Interface design, responsive layout and registration validation

The interface uses consistent reusable elements: a navigation bar, product cards, buttons, filtering controls, forms, basket content and short toast messages following user actions. React-Bootstrap is used for elements such as the navigation bar, the filtering accordion and layout containers, while custom CSS controls product grids, spacing, product detail views and basket presentation. This continues the simple Bootstrap-based visual style of the first mini-project but adapts it to reusable React components.

Responsiveness is implemented through a combination of Bootstrap layout behaviour and media queries.

The registration form uses Formik to control form state and Yup to validate the entered first name, last name, email and password before a registration request is submitted. Required-field error messages and invalid-email or name feedback are shown next to the relevant form fields. After successful registration, the saved profile details (full name, email, password, and an empty basket) are stored for use in a JSON file (`users.json`).

2 Design of the RESTful API

The backend was designed following RESTful API principles using Express.js. REST provides a simple way of structuring communication between the React frontend and the backend services. Each resource in the system is represented through dedicated endpoints that can be accessed using standard HTTP methods, which makes our API easier to maintain and extend.

2.1 Login and Registration endpoints

One of the challenges we came across while implementing basket-related operations had to do with managing baskets for registered users. We developed two endpoints to service the login and registration flow (`/users/login` and `/users/register`). This greatly improved both the user and the development experience since we could guarantee that changes resulting from a session would be stored, not on local storage, but on the storage solution that is used by our server. This allowed us to integrate both users and baskets.

2.2 Email and Username for login and registration

When creating the basket endpoints (`/user/:username/basket` and `/user/:username/basket/:productId`) we didn't have the user endpoint developed yet and we used only a username and a password for user details in the `users.json` file. However, due to the data validation requirement the client was subject to (email validation), we only require users to submit their email for registration, which we map to the `username` field in our server as our server is developed to save usernames, not emails. This allowed us to reuse an already existing endpoint, with already existing tests while incorporating data validation - as required - in the client.

2.3 Backend services

The implemented RESTful API is organized into three service areas that support the main functions of the webshop: accessing product information, managing user-specific shopping baskets, and handling user registration and login. These service areas are:

- **Products service:** Retrieve all products or filtered subsets by `color`, `gender`, or both; fetch a single product by `id`; list available categories (`gender`, `color`, `newArrival`, `discount`).
- **Basket service (per user):** Retrieve user basket or create guest user, add/merge items, remove all items, or delete a single item by `productId`.
- **Users service:** Register a new user and log in an existing user.

Figure 2 gives an overview of the implemented web service endpoints. The full API specifications are seen in appendix A.

Resource path	POST	GET	PUT	DELETE
/products/{id}		Retrieve the details for a specific product based on its {id}		
/products/categories		Retrieve all available categories and subcategories		
/products		Retrieve most important information about all products		
/products?color=?		Retrieve most important information about all products in certain color		
/products?gender=?		Retrieve most important information about all products for certain gender		
/products?color=?&gender=?		Retrieve most important information about all products for certain color and gender		
/user/{username}/basket		Get basket for a specific user based on their {username}.	Put an item to a specific user's basket	Remove all products from a specific user's basket
/user/{username}/basket/{productid}				Delete item from a specific user's basket
/users/login	Login with an existing user			
/users/register	Register a new user			

Figure 2: RESTful API specification with HTTP methods and paths

3 Software architecture of the developed system

3.1 Logical and physical architecture

The developed webshop follows a distributed client/server architecture composed of two independently running applications:

- a client-side single-page application developed in React and TypeScript
- a server-side RESTful API developed with Node.js and Express.

3.1.1 Logical architecture

From a logical perspective, the system is divided into presentation, application, and data-handling responsibilities.

The React client forms the presentation layer of the system. It is responsible for:

- rendering the user interface
- handling user interactions
- managing navigation between views
- displaying products and categories

- handling filtering operations
- managing basket interactions
- handling login and registration forms
- providing visual feedback to users

The Express server forms the application and service layer. It exposes RESTful endpoints which allow the client to:

- retrieve products and categories
- authenticate users and register new users
- manage baskets

The server also contains the business logic responsible for processing requests, validating operations, and interacting with stored JSON data.

The data layer is implemented through JSON-based persistence. Product and category information are stored in:

- `categories.json`
- `products.json`

While users and basket contents are stored in:

- `users.json`

This lightweight approach was appropriate for the scope of the project because it simplified development.

3.1.2 Physical architecture

Physically, the system is deployed as two separate runtime environments communicating through HTTP requests and JSON responses.

- The React client executes in the user's browser and typically runs locally on `localhost:3000`
- The Express REST API executes as a Node.js server on `localhost:3001`

Communication between the two applications occurs asynchronously through HTTP methods such as `GET`, `POST`, `PUT` and `DELETE`

Although both applications are submitted as a single software system, they operate as separate deployable components with clearly divided responsibilities.

3.1.3 Third-party components and frameworks

Several third-party libraries and frameworks were integrated to support development and improve maintainability.

Client-side technologies

- React provides component-based UI rendering
- TypeScript adds static typing and improves reliability and maintainability
- React Router enables client-side routing without full page reloads
- React Bootstrap and Bootstrap CSS provide responsive layouts and reusable UI styling
- Formik simplifies form state handling
- Yup performs schema-based form validation

Server-side technologies

- Node.js provides the JavaScript runtime environment
- Express.js provides routing, middleware handling, and API structure

These frameworks reduced boilerplate implementation work and encouraged modular and maintainable development practices.

3.2 Client-side React application architecture and patterns

The React application follows a modular component-based architecture. The system is divided into: pages, reusable UI components, custom hooks and context modules. This separation improves maintainability, readability, and reusability.

We used a Component-based design, and created the following reusable components:

- ProductCard
- ActionButton
- Filter
- FilterSwitch

- `BasketContent`
- `Toast` and
- `NavigationBar`

Moreover, even though we did not re-use every component we encapsulated specific UI and interaction logic into independent modules. This follows the React principle of separation of concerns by ensuring that each component has a focused responsibility. It also reduces duplication and allows interface changes to be implemented consistently across the application.

3.2.1 Single-page application (SPA) architecture

The client application follows the SPA architectural style. React Router dynamically changes the displayed view without reloading the browser page. This improves: responsiveness, navigation speed and user experience while allowing the navigation bar and shared UI elements to remain persistent across pages.

3.2.2 State management patterns

Several React state management techniques are used throughout the application:

Lifted state: State is lifted where necessary, for example when the all-products page controls the selected filters. This prevents duplicated state and keeps filtering behavior synchronized.

Context API: Authentication state is shared through `AuthContext`, making the current user and login/logout functionality available to the navigation bar and relevant pages. Authentication-related state is made available through `AuthContext`. The provider exposes the current user, login, saved user details and logout actions to components such as `NavigationBar` and the form pages. Local storage is used to retain safe user interface details and a generated guest identifier between page refreshes. This is a Context API solution rather than passing authentication data through every component level. The Context API avoids prop drilling by making the current user, login/logout functions, saved authentication details available globally to relevant components. This is more scalable than manually passing authentication props through multiple component layers.

3.2.3 Custom hooks and separation of logic

The application separates presentation from data handling through custom hooks to manage communication with the RESTful API. These are the hooks present in our project:

- `useProducts` obtains product lists and applies chosen filters by building the query string
- `useCategories` retrieves the available filter values from the server
- `useBasket` reads and modifies the basket and handles the guest identifier
- `useLogin` submits login information

This improves code reuse and keeps UI components focused primarily on rendering. Moreover, this approach follows the React pattern of abstraction through hooks and contributes to cleaner and more maintainable code.

3.2.4 Form handling and validation

The system uses Formik for form state management and Yup for declarative validation rules. This combination simplifies: input handling, validation logic as well as error management, while improving maintainability and reducing repetitive form code.

3.2.5 Responsive design principles

React Bootstrap and CSS support responsive layouts and reusable design components. This ensures that navigation, forms, product listings and basket layouts adapt appropriately to different screen sizes and device widths.

3.3 Server-side RESTful API architecture and patterns

3.3.1 Layered architecture

The server-side application follows a layered RESTful architecture. The Express application is organized into feature folders containing: routes, controllers and models. This structure follows separation of concerns principles.

- **Route layer:** maps URLs and HTTP methods, but also forwards requests to controllers

- **Controller layer:** processes request parameters and bodies. It also coordinates operations, and determines HTTP responses
- **Model layer:** contains business and data-access logic, reads and writes JSON data, also performs basket and user operations

Moreover, Express middleware is used to parse incoming JSON request bodies and support API request handling. Middleware centralises common processing logic and keeps route handlers cleaner.

3.3.2 Data storage solution

The system uses JSON files as a lightweight persistence mechanism instead of a relational or NoSQL database.

3.3.3 Modular backend design

As mentioned in Section 2.3 the backend uses feature-based modularisation where each domain area (products, users, baskets) is separated into independent modules. This modular structure: improves maintainability and simplifies debugging.

4 Use of generative AI in the project

Generative AI (ChatGPT/Codex/Claude) was used as a support tool during development and documentation, not as a replacement for the group's own design decisions or implementation responsibility. During the implementation work, it was used for suggestions concerning naming consistency, CSS organisation, debugging and explanations of code structure.

All descriptions of the final system should be reviewed against the submitted source code by the group before submission. The group remains responsible for verifying functionality, correcting errors and ensuring that the report accurately describes the final implementation.

A API Documentation

Path: /products/:id

Method: GET

Summary: Get all details about one specific product by id.

URL Params: product id (*int*)

Body of request: –

Success Response:

Code: 200 OK

Body Content: Product JSON object

```
{
  "id": 0,
  "name": "Classic Black Tee",
  "description": "A timeless black tee that pairs well with any
    outfit ...",
  "image": "../images/tshirt1.png",
  "discount": 0.3,
  "newArrival": false,
  "price": 24.99,
  "color": "Black",
  "gender": "Women"
}
```

Error Response:

Code: 404 NOT FOUND

Body Content: { "error": "Product not found" }

Code: 400 BAD REQUEST

Body Content: { "error": "Invalid request" }

Sample Call:

```
let response = await fetch('
  http://localhost:3001/products', {
    method: 'GET',
    headers: { 'Content-Type': 'application/json;charset=utf-8' }
  });

let data = await response.json();

let productId = 0;
```

```
let response = await fetch('http://localhost:3001/products/${productId}',
  { method: 'GET',
    headers: {
      'Content-Type': 'application/json;charset=utf-8' }
  });
let data = await response.json();
```

Path: /user/:username/basket

Method: GET

Summary: Get shopping basket for a specific user.

URL Params: username

Body of request: –

Success Response:

Code: 200 OK

Example: GET http://localhost:3001/user/mia.anderson/basket

HTTP/1.1

Body Content: Array of username, basket items and their fields, total basket price

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 248
ETag: W/"f8-k4nJQIipXLzqKksn4ECpJRUIfso"
Date: Thu, 02 Apr 2026 17:42:26 GMT
Connection: close
```

```
{
  "username": "mia.anderson",
  "basket": [
    {
      "productId": 0,
      "productName": "Classic Black Tee",
      "quantity": 1,
      "unitPrice": 24.99,
      "discount": 0.3
    },
    {
```

```
    "productId": 5,  
    "productName": "Sunny Yellow Tee",  
    "quantity": 2,  
    "unitPrice": 22.99,  
    "discount": 0  
  }  
],  
  "totalPrice": "63.47"  
}
```

Error Response:**Code:** 404 NOT FOUND**Example:** GET http://localhost:3001/user/wrong.name/basket HTTP/1.1**Body Content:**

```
HTTP/1.1 404 Not Found  
X-Powered-By: Express  
Content-Type: application/json; charset=utf-8  
Content-Length: 30  
ETag: W/"1e-6mMVRLQyQddjCc6b2P2Pfr6E460"  
Date: Thu, 02 Apr 2026 17:40:33 GMT  
Connection: close  
  
{  
  "error": "username not found"  
}
```

Sample Call:

```
let response = await  
  fetch('http://localhost:3001/user/mia.anderson/basket', {  
    method: 'GET',  
    headers: { 'Content-Type': 'application/json; charset=utf-8' },  
    body: JSON.stringify(basket)  
  });  
  
let data = await response.json();
```

Method: PUT

Summary: Put an item to a specific user's basket. If the item already exists, its quantity is increased.

URL Params: username

Body of request: A basket item in a pair format [productId, quantity]

```
{
  "product": [0, 2]
}
```

Success Response:

Code: 200 OK

Example: PUT http://localhost:3001/user/mia.anderson/basket HTTP/1.1

Body Content: Array of username, basket items and their quantities

```
{
  "username": "mia.anderson",
  "basket": [
    {
      "productId": 0,
      "productName": "Classic Black Tee",
      "quantity": 12,
      "unitPrice": 24.99,
      "discount": 0.3
    },
    {
      "productId": 5,
      "productName": "Sunny Yellow Tee",
      "quantity": 2,
      "unitPrice": 22.99,
      "discount": 0
    }
  ],
  "totalPrice": "255.90"
}
```

Error Response:

Code: 404 Not found

Example: PUT http://localhost:3001/user/dummy.user/basket HTTP/1.1

Body Content:

```
HTTP/1.1 404 Not Found
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 36
```

```
ETag: W/"24-dvNS5NNEZ2fufMFCrLgdCtO4LiA"
Date: Thu, 09 Apr 2026 14:48:31 GMT
Connection: close

{
  "error": "Basket or user not found"
}
```

Sample Call:

```
let product = { product: [0, 2] };

let response = await
  fetch('http://localhost:3001/user/mia.anderson/basket', {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json;charset=utf-8' },
    body: JSON.stringify(product)
  });
let data = await response.json();

let response = await
  fetch('http://localhost:3001/user/dummy.user/basket', {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json;charset=utf-8' },
    body: JSON.stringify(product)
  });
let data = await response.json();
```

Method: DELETE

Summary: Remove all products from a specific user's basket

URL Params: username

Body of request: –

Success Response:

Code: 200 OK

Body Content:

```
{
  "message": "Order placed successfully",
  "basket": {
    "username": "ethan.diaz",
    "basket": [],
  }
}
```

```
    "totalPrice": "0.00"
  }
}
```

Error Response:

Code: 404 NOT FOUND

Body Content: { "error": "Basket not found" }

OR { "error": "username not found" }

Code: 400 BAD REQUEST

Body Content: "{error message}" (in case of an error message returned by the model)

Sample Call:

```
const response = await
  fetch("http://localhost:3001/user/johndoe/basket/3", {
    method: "DELETE",
    headers: {
      "Content-Type": "application/json;charset=utf-8"
    }
  });

const result = await response.json();
console.log(result);
```

Path: /user/:username/basket/:productId

Method: DELETE

Summary: Delete item from a specific user's basket

URL Params: username and productId

Body of request: –

Success Response:

Code: 200 OK

Body Content:

```
[
  {
    "fname": "Mia",
    "lname": "Anderson",
    "username": "mia.anderson",
    "password": "MiaPass123!",
```

```
    "basket": [
      [
        0,
        6
      ],
      [
        5,
        2
      ]
    ]
  }
]
```

Error Response:

Code: 404 Not found

Body Content: { "error": "Product not found in basket" }

OR { "error": "User not found" }

Sample Call:

```
const response = await
  fetch("http://localhost:3001/user/johndoe/basket/3", {
    method: "DELETE",
    headers: {
      "Content-Type": "application/json;charset=utf-8"
    }
  });

const result = await response.json();
console.log(result);
```

Path: /products/categories

Method: GET

Summary: Retrieve all available categories and their unique values (gender, color, newArrival, discount)

URL Params: –

Body: –

Success Response:

Code: 200 OK

Example: GET http://localhost:3001/products/categories HTTP/1.1

Body Content:

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 153
ETag: W/"99-wFCS2T+zpQUVbiWI6CFsSjqB5Tk"
Date: Thu, 02 Apr 2026 17:47:17 GMT
Connection: close
```

```
{
  "gender": [
    "Women",
    "Men"
  ],
  "color": [
    "Black",
    "Red",
    "Blue",
    "White",
    "Purple",
    "Yellow",
    "Orange",
    "Green"
  ],
  "newArrival": [
    false,
    true
  ],
  "discount": [
    0.3,
    0.2,
    0.5,
    0
  ]
}
```

Error Response:

Code: 404 Not found

Example: GET http://localhost:3001/products/categories HTTP/1.1

In the case of no product fields having been set as categories in categories.json

Body Content:

```
HTTP/1.1 404 Not Found
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 33
ETag: W/"21-a2GBY3sBMtZq7UgWGHjbJtDwxzU"
Date: Thu, 02 Apr 2026 17:52:57 GMT
Connection: close

{
  "error": "No categories are set"
}
```

Sample Call:

```
let response = await fetch('http://localhost:3001/products/categories', {
  method: 'GET',
  headers: { 'Content-Type': 'application/json; charset=utf-8' },
  body: JSON.stringify(basket)
});

let data = await response.json();
```

Path: /products

Method: GET

Summary: Retrieve most important information of all products.

URL Params: –

Body of request: –

Success Response:

Code: 200 OK

Body Content: Array of product JSON objects

```
[
  {
    "price": 24.99,
    "discount": 0.3,
    "color": "Black",
    "name": "Classic Black Tee",
    "id": 0,
```

```
[
  "gender": "Women"
},
...
]
```

Error Response:

Code: 400 BAD REQUEST

Body Content: { "error": "Couldn't read products data" }

Sample Call:

```
const response = await fetch("http://localhost:3001/products", {
  method: "GET",
  headers: {
    "Content-Type": "application/json;charset=utf-8"
  }
});
const products = await response.json();
console.log(products);
```

Path: /products?color=?

Method: GET

Summary: Retrieve most important information of all products of a certain colour.

URL Params: color

Body of request: –

Success Response:

Code: 200 OK

Body Content: Array of product JSON objects

```
[
  {
    "price": 24.99,
    "discount": 0.3,
    "color": "Black",
    "name": "Classic Black Tee",
    "id": 0,
    "gender": "Women"
  },
]
```

Error Response:**Code:** 400 BAD REQUEST**Body Content:** { "error": "Couldn't read products data" }**Sample Call:**

```
const response = await
  fetch("http://localhost:3001/products?color=black", {
    method: "GET",
    headers: {
      "Content-Type": "application/json;charset=utf-8"
    }
  });
const products = await response.json();
console.log(products);
```

Path: /products?gender=?**Method: GET****Summary:** Retrieve most important information of all products for a certain gender**URL Params:** gender**Body of request:** –**Success Response:****Code:** 200 OK**Body Content:** Array of product JSON objects

```
[
  {
    "price": 24.99,
    "discount": 0.3,
    "color": "Black",
    "name": "Classic Black Tee",
    "id": 0,
    "gender": "Women"
  },
]
```

Error Response:**Code:** 400 BAD REQUEST**Body Content:** { "error": "Couldn't read products data" }

Sample Call:

```
const response = await
  fetch("http://localhost:3001/products?gender=women", {
    method: "GET",
    headers: {
      "Content-Type": "application/json;charset=utf-8"
    }
  });
const products = await response.json();
console.log(products);
```

Path: /products?gender=?&color=?

Method: GET

Summary: Retrieve most important information of all products for certain gender and color. The endpoint also accepts passing of the same filtering condition with several different values, i.e.:

GET http://localhost:3001/products?gender=Men&color=Blue&color=Red
HTTP/1.1

URL Params: gender and/or color

Body of request: –

Success Response:

Code: 200 OK

Body Content: Array of product JSON objects

```
[
  {
    "price": 29.99,
    "discount": 0,
    "color": "Blue",
    "description": "Make a splash with our Trendy Teal Tee! This vibrant teal shirt is made from soft, breathable cotton for all-day comfort. Perfect for adding a cool pop of color to your wardrobe, it's a must-have for anyone who loves to make a statement with their style.",
    "name": "Trendy Teal Tee",
    "id": 7,
    "gender": "Men",
    "image": "/images/tshirt8.png",
```

```

    "newArrival": false
  },
  {
    "price": 26.99,
    "discount": 0,
    "color": "Red",
    "description": "Add a vintage vibe to your wardrobe with our Washed Red Tee! This faded red shirt is made from soft, breathable cotton for all-day comfort. Perfect for adding a worn-in pop of color to your wardrobe, it's a must-have for anyone who loves to make a statement with their style.",
    "name": "Washed Red Tee",
    "id": 12,
    "gender": "Men",
    "image": "/images/tshirt13.png",
    "newArrival": true
  }
]

```

Error Response:

Code: 400 BAD REQUEST

Body Content: { "error": "Couldn't read products data" }

Sample Call:

```

const response = await
  fetch("http://localhost:3001/products?color=black&gender=woman",
    {
      method: "GET",
      headers: {
        "Content-Type": "application/json;charset=utf-8"
      }
    });
const products = await response.json();
console.log(products);

```

Path: /users/login

Method: POST

Summary: Login with an existing user from `server/API/users/users.json`

URL Params: –

Body of request: email and password data

```
POST http://localhost:3001/users/login HTTP/1.1
Content-Type: application/json

{
  "email": "migi@inpay.com",
  "password": "1234"
}
```

Success Response:

Code: 200 OK

Body Content:

```
{
  "firstName": "miguel",
  "lastName": "angel",
  "email": "migi@inpay.com"
}
```

Error Response:

Code: 400 BAD REQUEST

Body Content: "error": "Email and password are required"

Code: 401 UNAUTHORIZED

Body Content: "error": "Invalid password"

Code: 404 NOT FOUND

Body Content: "error": "User not found"

Code: 400 BAD REQUEST

Body Content: "{error message}" (in case of an error message returned by the model)

Sample Call:

```
POST http://localhost:3001/users/login HTTP/1.1
Content-Type: application/json

{
  "email": "migi@inpay.com",
  "password": "1234"
}

### returns 404 not found — "User not found"
POST http://localhost:3001/users/login HTTP/1.1
Content-Type: application/json
```

```

{
  "email": "migi22@inpay.com",
  "password": "1234"
}

### returns 401 not authorised – "Invalid password"
POST http://localhost:3001/users/login HTTP/1.1
Content-Type: application/json

{
  "email": "migi@inpay.com",
  "password": "123422"
}

### returns 400 bad request – "Email and password are required"
POST http://localhost:3001/users/login HTTP/1.1
Content-Type: application/json

{
  "email": "",
  "password": ""
}

```

Path: /users/register

Method: POST

Summary: Register a new user (as long as the username is not present in server/API/users/users.json)

URL Params: none

Body: firstname, lastname, email and password

For example:

```

{
  "firstName": "Test",
  "lastName": "User",
  "email": "test.user@example.com",
  "password": "TestPass123!"
}

```

Success Response:**Code:** 201 CREATED**Body Content:**

```
{
  "fname": "Test",
  "lname": "User",
  "username": "test.user@example.com",
  "password": "TestPass123!",
  "basket": []
}
```

Error Response:**Code:** 400 BAD REQUEST**Body Content:** "All fields are required"**Code:** 400 BAD REQUEST**Body Content:** "{error message}" (in case of an error message returned by the model)**Code:** 409 BAD REQUEST**Body Content:** "User already exists"**Sample Call:**

```
POST http://localhost:3001/users/register HTTP/1.1
Content-Type: application/json
```

```
{
  "firstName": "Test1",
  "lastName": "User",
  "email": "test.user22@example.com",
  "password": "TestPass123!"
}
```